

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Гилязова Алина

Группа J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2023 г.

## Задача

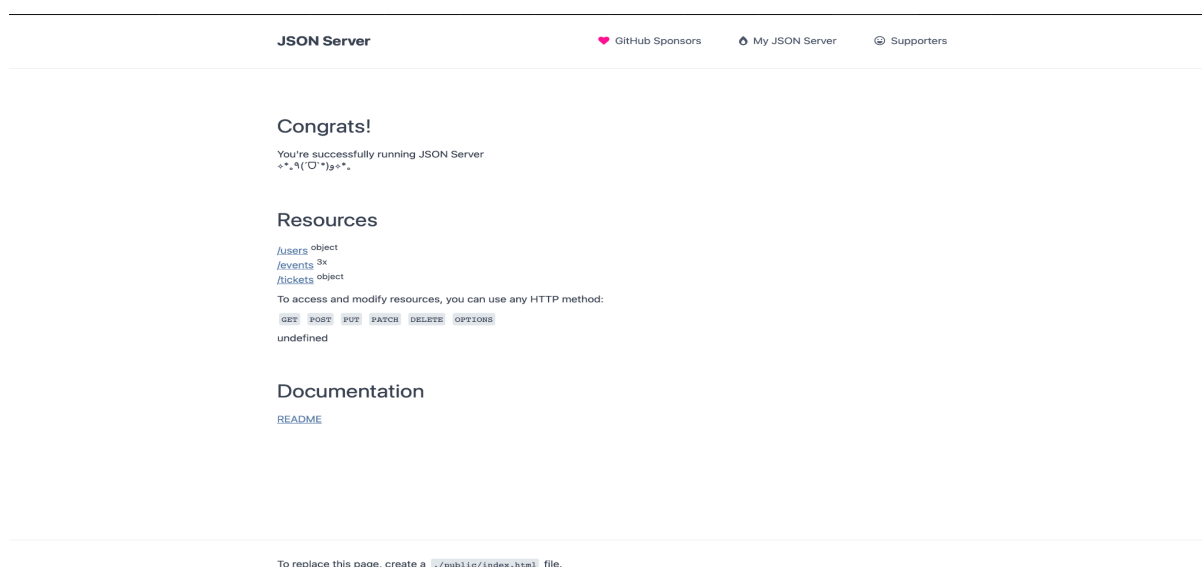
Варианты остаются прежними. Теперь Вам нужно привязать то, что Вы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализуйте моковое API средствами JSON-сервера и подключите к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Имитация работы с API".

Например, для приложения для просмотра прогнозов погоды задание выглядит следующим образом:

Реализовать получение погоды (прогноз на ближайшие 7 дней) из открытого API OpenWeatherMap, в зависимости от геолокации пользователя. Реализовать вывод полученного прогноза в виде 7 карточек в три ряда (первый ряд - крупная карточка, второй ряд - три карточки в меньшем размере, третий ряд - четыре карточки в маленьком размере).

## Ход работы

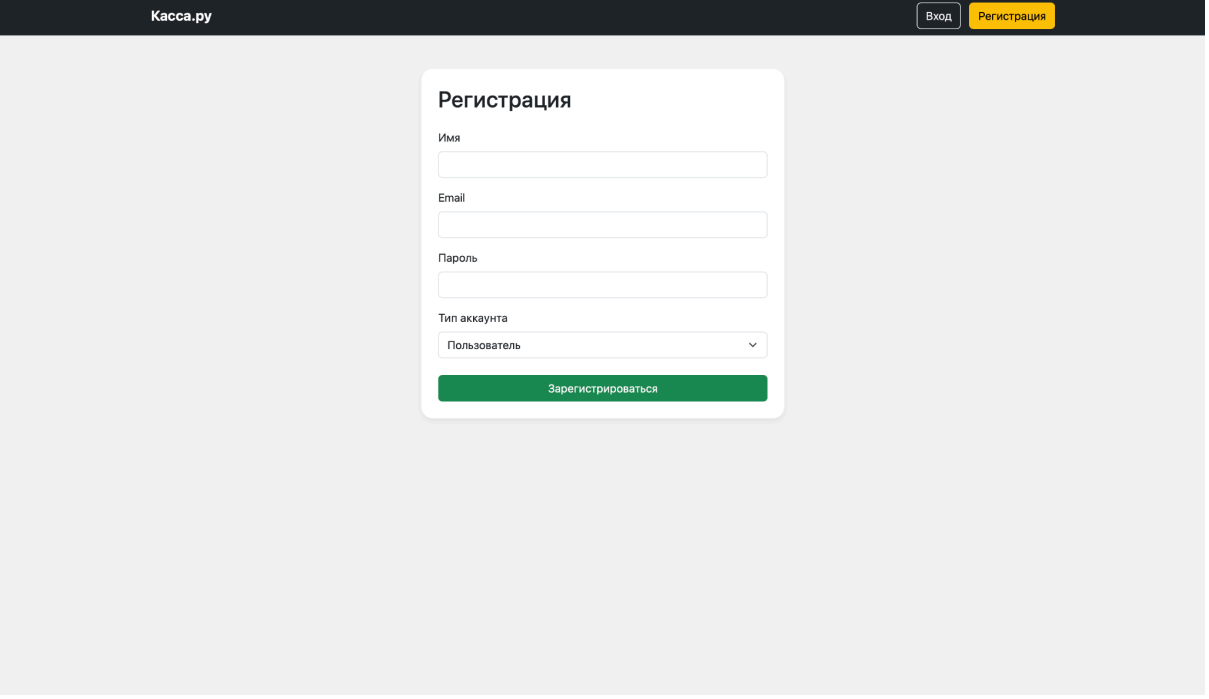
В ходе выполнения лабораторной работы был доработан веб-проект платформы для покупки билетов, разработанный в предыдущей лабораторной работе. Основной задачей данной работы было подключение интерфейса к внешнему API и организация взаимодействия клиента с сервером.



Для реализации серверной части был использован инструмент JSON-server, позволяющий быстро создать моковое API на основе файла

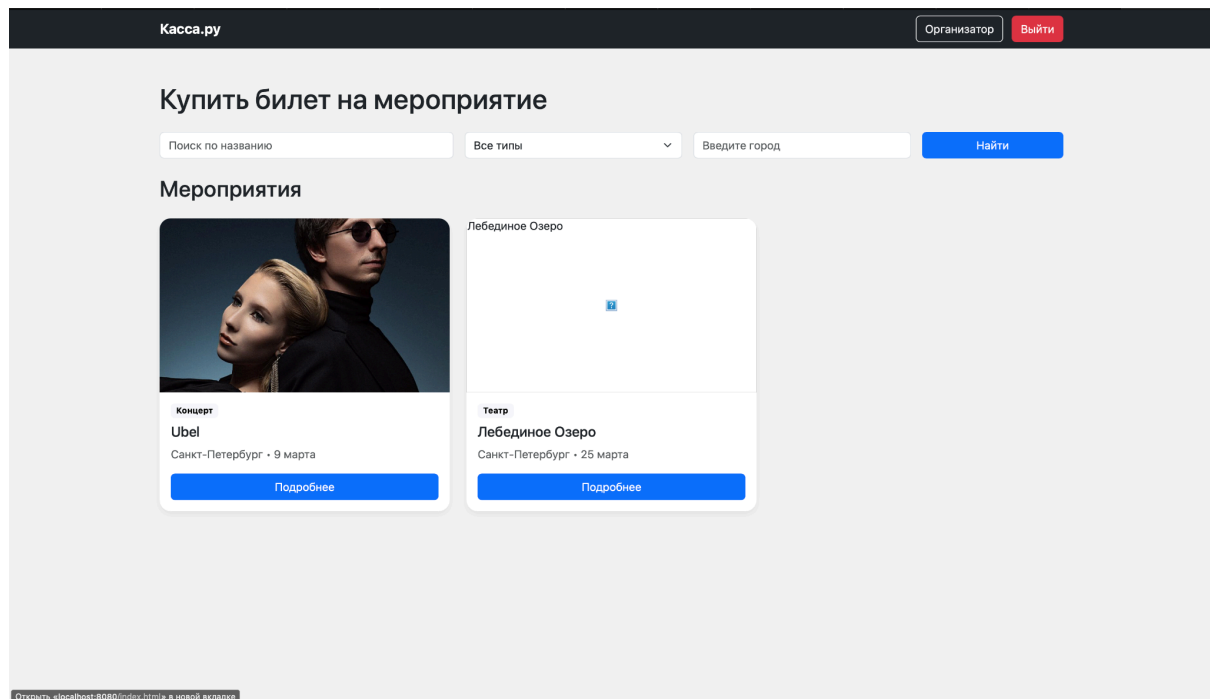
базы данных db.json. В этом файле хранятся основные сущности приложения: пользователи, мероприятия и билеты. JSON-server автоматически предоставляет REST-интерфейс для работы с этими данными.

В проект была добавлена авторизация пользователей. Для этого реализованы формы регистрации и входа, через которые данные пользователя отправляются на сервер с помощью HTTP-запросов. После успешного входа сервер возвращает токен авторизации и информацию о пользователе. Эти данные сохраняются на стороне клиента и используются для выполнения последующих запросов к API.

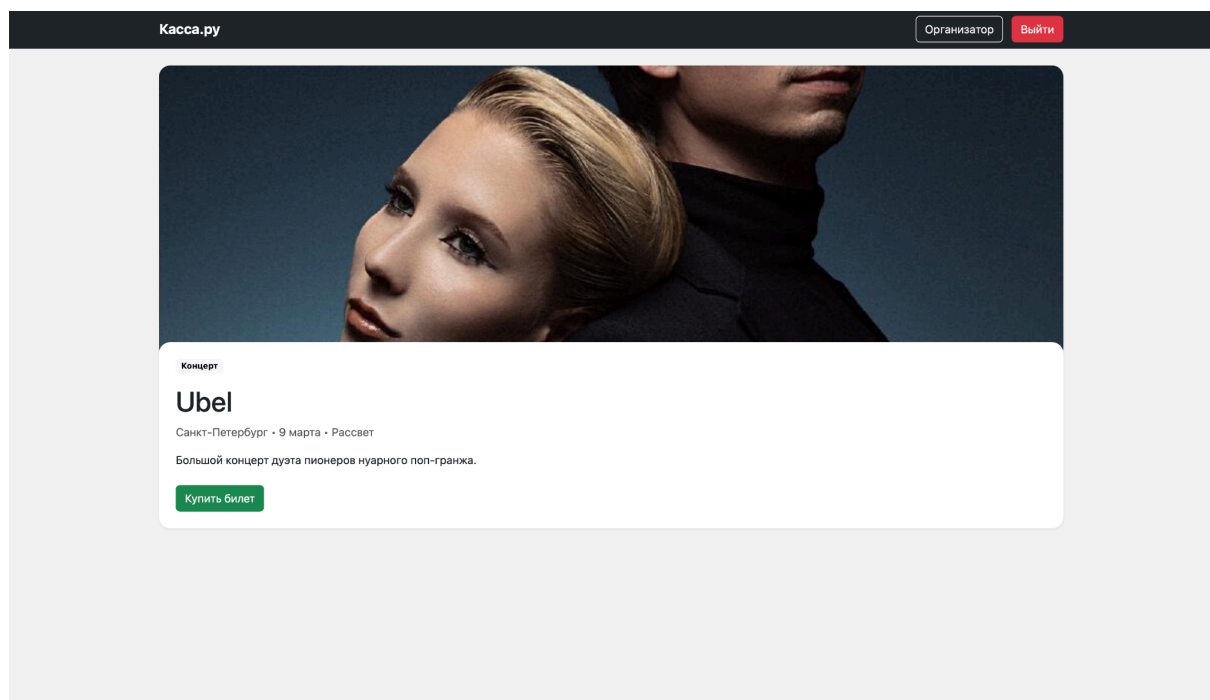


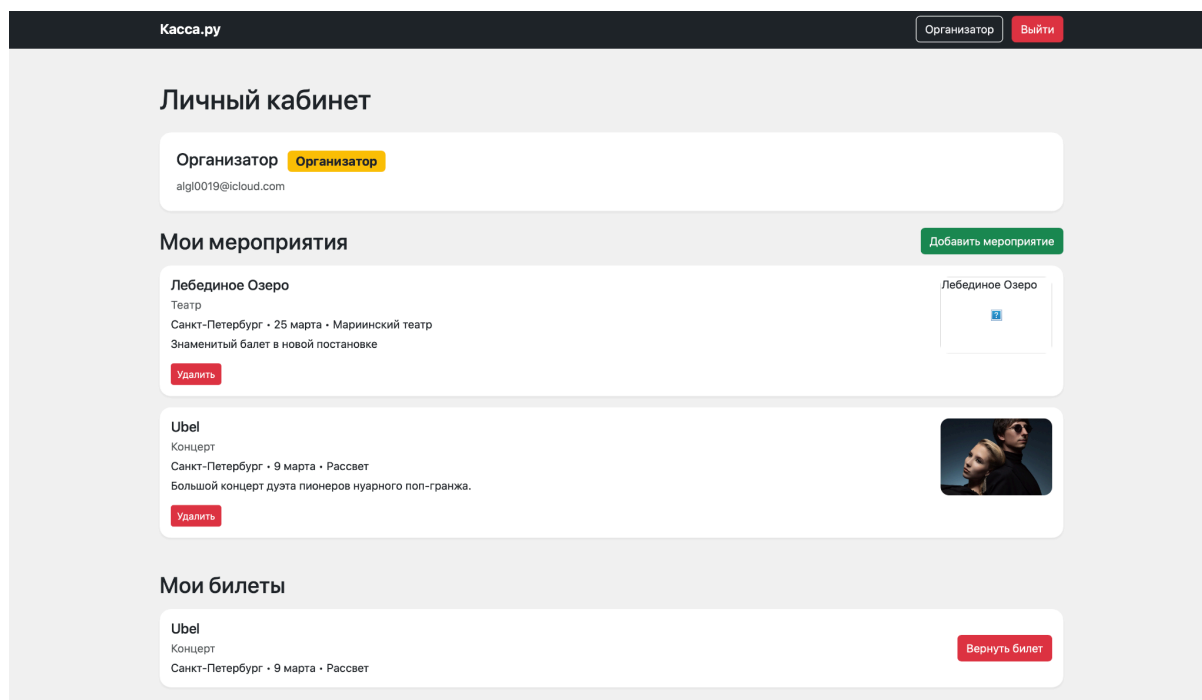
The screenshot shows a web application interface for 'Касса.ру' (Kassa.ru). At the top, there is a dark header with the site name on the left and two buttons, 'Вход' (Login) and 'Регистрация' (Registration), on the right. The 'Регистрация' button is highlighted in yellow. Below the header, the main content area is light gray and contains a white registration form titled 'Регистрация'. The form includes input fields for 'Имя' (Name), 'Email', and 'Пароль' (Password). Below these is a dropdown menu for 'Тип аккаунта' (Account type) with 'Пользователь' (User) selected. At the bottom of the form is a green button labeled 'Зарегистрироваться' (Register).

Для взаимодействия с сервером в приложении используется функция `fetch`. С её помощью реализованы основные операции работы с данными. Получение списка мероприятий выполняется через GET-запрос к серверу. После получения данных интерфейс автоматически обновляется и отображает список доступных мероприятий в виде карточек.

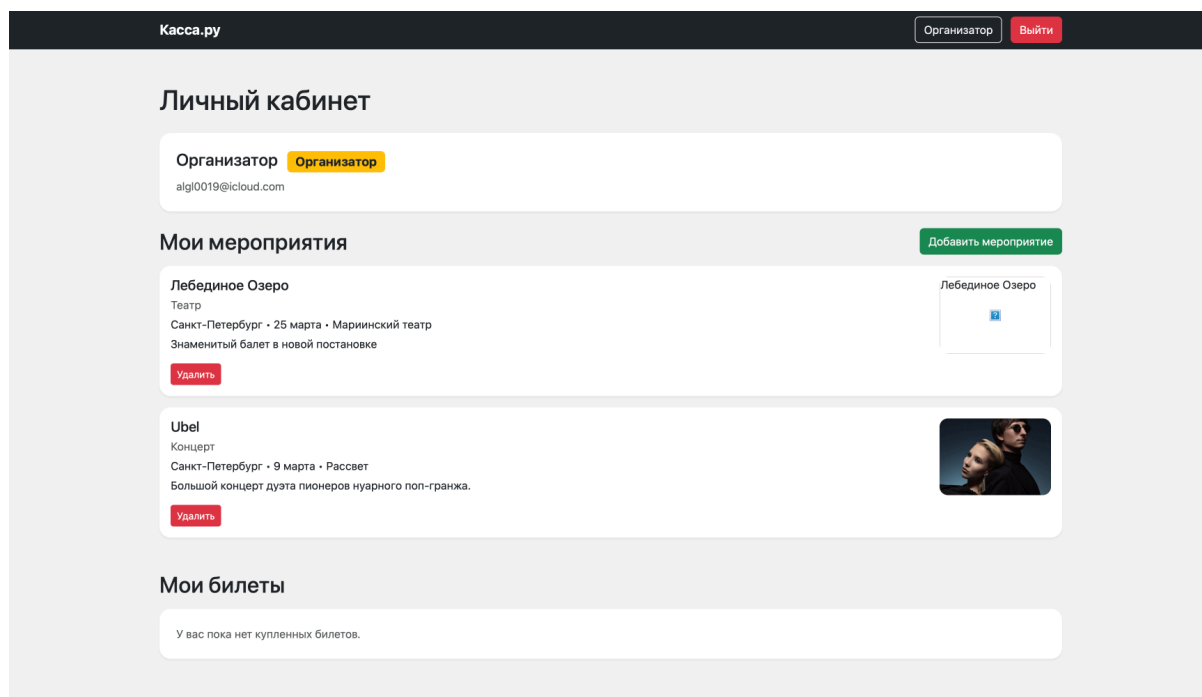


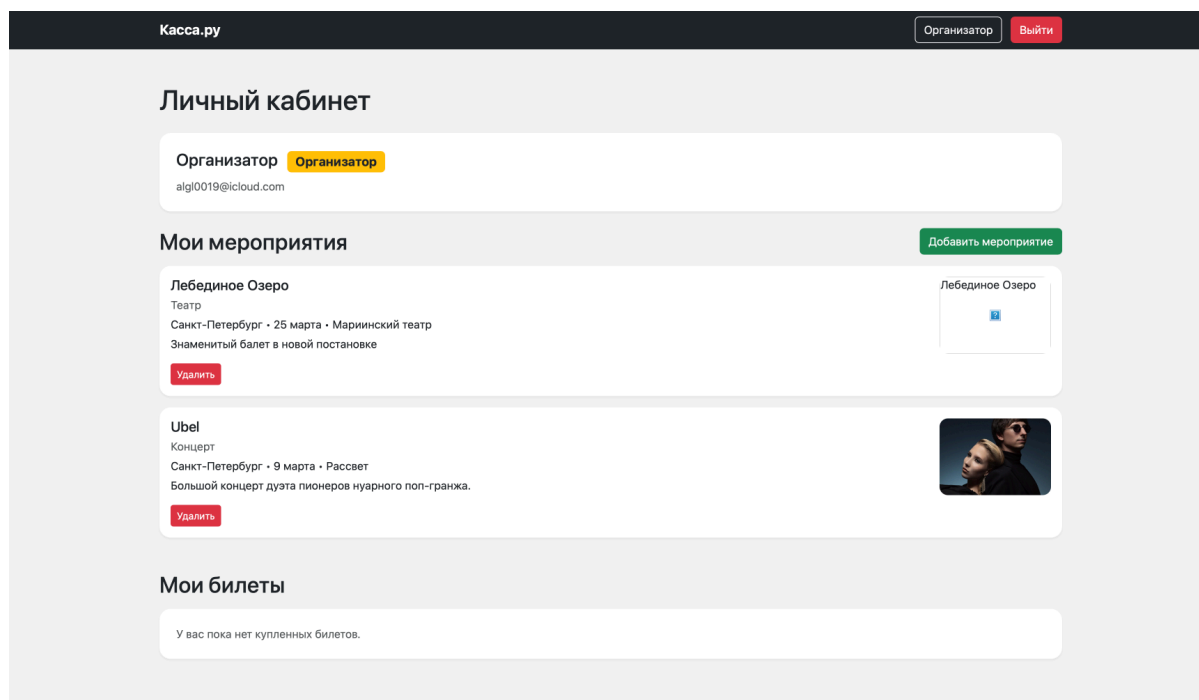
При покупке билета отправляется POST-запрос на сервер для создания новой записи о билете. При возврате билета выполняется запрос на изменение или удаление соответствующей записи. Таким образом, все операции пользователя напрямую взаимодействуют с серверной частью приложения.





Также была реализована возможность добавления мероприятий организатором. При создании нового мероприятия данные формы отправляются на сервер через POST-запрос и сохраняются в базе данных. После этого мероприятие автоматически появляется в общем списке событий.





Кроме того, в проекте была реализована загрузка данных с сервера при открытии страниц. Благодаря этому интерфейс всегда отображает актуальную информацию, которая хранится в базе данных JSON-server.

Таким образом, интерфейс веб-приложения был интегрирован с внешним API, а хранение данных было перенесено с локального хранилища браузера на серверную часть.

## Вывод

В результате выполнения лабораторной работы был доработан прототип веб-платформы для покупки билетов и реализовано взаимодействие интерфейса с внешним API. Для имитации серверной части был использован JSON-server, позволивший создать REST-интерфейс для работы с данными.

В ходе работы были получены практические навыки работы с HTTP-запросами, использованием функции fetch для взаимодействия с API, а также организации авторизации пользователей и обработки данных, получаемых с сервера. Реализованная система позволяет выполнять основные операции с мероприятиями и билетами через API, что приближает приложение к реальной архитектуре веб-сервисов.